Fuente de la verdad para inicios de proyectos

Guía ejecutiva de inicio de proyectos con IA en Iberpixel

> **Objetivo**
> Este documento es la **fuente de verdad** para iniciar y trabajar un proyecto de desarrollo con agentes de IA en Iberpixel.
> Define cómo usamos:

- Spec-Driven Development (SDD)
- TDD "a lo agents"
- Skills y comandos de IA
- Estructura de proyecto y carpetas
- Buenas prácticas con IA, Git y Jira
- **Trazabilidad obligatoria de ejecución y verificación asistida por IA**

1. Principios básicos

1. **La IA no diseña, ejecuta**
   - El diseño, la arquitectura y las decisiones técnicas son responsabilidad del developer.
   - La IA se usa para implementar, refactorizar, testear y documentar **decisiones ya tomadas y especificadas**.
2. **Spec-Driven Development (SDD)**
   - Ningún trabajo significativo se hace sin una **spec previa**.
   - Cada ticket relevante en Jira tiene su spec en `.ai/specs/`.
   - La spec define comportamiento, restricciones y criterios de aceptación (AC).
3. **TDD "a lo agents"**
   - Primero se define qué significa "estar bien hecho" (AC + plan de verificación).
   - Luego se implementa.
   - Después se verifica automáticamente con `verify` (Playwright / API / etc.).
4. **AGENTS.md como contrato de proyecto**
   - Cada repo tiene un `AGENTS.md` que las herramientas de IA deben leer siempre.
   - Ahí viven:
     - Stack tecnológico
     - Convenciones (código, carpetas, Git)
     - Comandos del proyecto
     - Skills y cómo usarlos
5. **Trazabilidad IA-asistida**
   - Toda PR debe:
     - Referenciar el ticket de Jira.
     - Referenciar la spec correspondiente.
     - Indicar qué partes han sido IA-asistidas.
   - **Toda ejecución IA-asistida que afecte a código debe dejar rastro en la spec** del ticket:

- Sección de resultado de ejecución.
- Relación con verificación (`verify`).
- Estado de los AC tras la ejecución.

2. Estructura mínima del repositorio

Todo proyecto que use IA en Iberpixel debe incluir **como mínimo**:

```
proyecto/
├── README.md
├── AGENTS.md                 # Reglas universales del proyecto
├── CLAUDE.md                 # (Recomendado) Config específica para Claude Code
├── .ai/
│   ├── specs/                # Specs por ticket Jira
│   │   └── <KEY>-spec.md
│   ├── verify/               # Config y scripts de verificación automática
│   │   └── config.yaml
│   ├── skills/               # Skills de IA específicos del proyecto
│   │   └── <nombre-skill>.md
│   └── prompts/              # (Opcional) Guías de estilo / prompts compartidos
├── .claude/
│   ├── settings.json         # Permisos, exclusiones, config del workspace
│   └── commands/             # Comandos reutilizables (wrappers de skills)
├── .mcp.json                 # (Si aplica) Config de servidores MCP
├── .gitignore
└── src/                      # Código de la aplicación (según stack)
```

`.gitignore` debe excluir como mínimo:

```
CLAUDE.local.md
.claude/settings.local.json
.env
.env.*
```

> Referencias:

- Índice del framework de IA:
  Framework de desarrollo con IA — Índice y punto de entrada
- Documento de kickoff (fundamentos SDD):
  Framework de desarrollo con IA — Documento de kickoff (EOS-244)
- Guía general de agentes de IA en desarrollo:
  https://iberpixel.atlassian.net/wiki/spaces/HO/pages/1033404418 Request access

3. Flujo estándar: Jira → Spec → Código → Execution Result → Verify → PR

Este es el flujo **por defecto** para cualquier feature o cambio relevante:

1. **Ticket en Jira (** `EOS-XXX` o proyecto correspondiente)
   - Historia de usuario / tarea técnica.
   - Criterios de aceptación básicos (AC).
2. **Crear spec técnica**
   - Fichero: `.ai/specs/<KEY>-spec.md` (ej: `.ai/specs/EOS-244-spec.md`)
   - Usar siempre la plantilla oficial de spec:
     Plantilla de SPEC + Criterios de Aceptación (para features con agentes)
   - Contenido mínimo:

    - Contexto
    - Objetivo
    - Restricciones técnicas
    - Casos límite
    - Criterios de aceptación (AC-1, AC-2, ...)
    - Criterio de “done”
3. **Crear rama Git**
    - Desde `main` (o rama trunk): `KEY-descripcion-corta`
        - Ejemplo: `EOS-244-spec-sdd-kickoff`
    - La spec debe existir **antes del primer commit** de implementación.
4. **Configurar la sesión con la IA**
    - Siempre que se use la IA:
        - Incluir `AGENTS.md` como contexto base.
        - Incluir la spec del ticket actual (`.ai/specs/<KEY>-spec.md`).
    - Para Claude Code:
        - `CLAUDE.md` debe indicar que `AGENTS.md` es el contrato principal del repo.
5. **Implementación guiada por la spec**
    - La IA no inventa requisitos → sigue los AC de la spec.
    - El dev:
        - Revisa y entiende todo el código propuesto.
        - Ajusta el diseño si ve problemas (la spec manda).
    - **Todo cambio relevante debe estar trazado contra AC concretos**:
        - Idealmente: “Este bloque implementa AC-2 y AC-3”.

5b. **Actualizar spec con resultado de ejecución (Execution Result obligatorio)**

- Tras implementar y ejecutar las pruebas relevantes (manuales o automáticas), el agente o el dev **deben** añadir al final de `.ai/specs/<KEY>-spec.md` una sección:

```
## Execution Result

- Fecha de implementación: YYYY-MM-DD HH:MM (zona horaria del equipo)
- Estado de AC:
  - AC-1: PASS | FAIL – nota breve
  - AC-2: PASS | FAIL – nota breve
  - ...
- Ficheros creados o modificados:
  - `ruta/archivo1.ext`
  - `ruta/archivo2.ext`
- verify:
  - Comando ejecutado: `npm run verify` / `./scripts/verify.sh <KEY>` / otro
  - Resultado: PASS | FAIL – descripción breve
- AI-assisted:
  - Herramienta(s): Claude / otro
  - Alcance: qué partes del trabajo se hicieron con IA
```

- Reglas operativas:
    - Esta sección es **obligatoria** para cualquier trabajo IA-asistido que toque código.
    - La sección `## Execution Result` se **actualiza** si hay nuevas ejecuciones relevantes (no se borra historial; se puede añadir subapartados con fecha).
    - La información de `verify` debe ser coherente con la configuración en `.ai/verify/config.yaml`.

- Reglas para agentes:
  - Los agentes de IA **no pueden considerar una tarea como terminada** (`finish` en canvas o equivalente) sin:
    i. Haber **verificado** la implementación conforme a la spec (al menos a nivel de `verify` o pruebas acordadas).
    ii. Haber **actualizado** la spec con la sección `## Execution Result` o haber completado y actualizado la sección existente.
  - Si por cualquier motivo el agente **no puede ejecutar verify**, debe:
    - Declararlo explícitamente en `## Execution Result` indicando la causa.
    - Marcar los AC correspondientes como "NO VERIFICADO" y **no** marcar la tarea como completamente terminada.
- Reglas para reviewers:
  - Esta sección es **lectura obligatoria** del reviewer antes de aprobar la PR.
  - La PR es **rechazable** si:
    - No existe la sección `## Execution Result`.
    - La sección está incompleta (AC sin estado, verify sin resultado, etc.).
    - Lo descrito ahí no cuadra con el diff real o con el pipeline de CI.

1. **Verificación automática (** `verify`)
   - Config central en `.ai/verify/config.yaml` (ver ejemplo oficial:
     🧩 Ejemplo listo para desplegar de config.yaml
   - Se ejecuta con el comando propio del proyecto (ejemplo):
     `npm run verify` o `./scripts/verify.sh <KEY>`
   - No se abre PR mientras haya AC en FAIL.
   - Cuando `verify` se ejecuta:
     - El resultado debe quedar reflejado en `## Execution Result` de la spec.
     - Si se generan artefactos de verificación (logs, capturas, reportes), se enlazan desde la spec o desde la PR según convenga.
2. **Pull Request**
   La PR debe incluir:

- Referencia al ticket Jira (`KEY`).
- Referencia a la spec (`.ai/specs/<KEY>-spec.md`).
- Estado de verificación (`verify` en verde).
- Indicación de código IA-asistido (ej: etiqueta, sección en la descripción o prefijos en commits).
- Confirmación explícita de que la spec correspondiente:
  - Existe.
  - Está actualizada.
  - Incluye una sección `## Execution Result` coherente con el trabajo realizado.

> **Regla de oro:**
> Una PR sin spec previa, sin relación clara con su spec o con la spec **desactualizada respecto al código y verify** es **rechazable**.

4. Spec-Driven Development (SDD)

4.1. Qué es una spec válida

Una spec es un **contrato ejecutable**, no un documento “de relleno”.
Debe responder, como mínimo, a:

1. **Contexto**
   - Dónde estamos dentro del sistema.
   - Qué ya existe.
   - Qué límites hay (tecnológicos, de negocio, UX, etc.).
2. **Objetivo**
   - Qué comportamiento nuevo o cambio concreto se quiere.
   - En términos de usuario / sistema, no de implementación.
3. **Restricciones**
   - Qué **no** puede ocurrir:
     - Patrones prohibidos (ej: no lógica de negocio en componentes de UI).
     - Dependencias no aceptadas.
     - Límites de rendimiento, accesibilidad, seguridad, etc.
4. **Casos límite**
   - Inputs extraños, estados intermedios, errores esperables.
   - Qué se espera que suceda en cada uno.
5. **Criterios de aceptación (AC)**
   - Lista numerada: AC-1, AC-2, …
   - Cada AC debe ser:
     - Binario (cumple / no cumple).
     - Observable (por UI, API, logs, etc.).
     - Verificable automáticamente (idealmente por `verify`).
6. **Criterio de “done”**
   - Condición para considerar cerrada la tarea:
     - Todos los AC en PASS.
     - `verify` en verde.
     - Documentación mínima (ej: actualización en `AGENTS.md` si cambia algo global).
     - **Spec actualizada con la sección** `## Execution Result`.

4.2. Buenas prácticas SDD

Hacer:

- Escribir/ajustar la spec **antes** de pedir nada a la IA.
- Ser concreto en restricciones (qué se permite y qué no).
- Versionar la spec en Git como parte normal del desarrollo.
- Referenciar la spec desde la PR y en comentarios de código cuando aplique.
- **Tratar la sección** `## Execution Result` como parte del contrato, no como apéndice opcional.

No hacer:

- Pedir a la IA “hazme la feature” sin spec.
- Dejar que la IA cambie specs sin validación humana.
- Usar specs como simple check-list de UI, sin casos límite ni restricciones.
- Considerar “terminada” una subtarea IA-asistida si la spec no refleja lo que realmente se ejecutó y verificó.

5. TDD “a lo agents”

Referencia ampliada:
Integración de TDD "a lo agents" en el desarrollo de software con IA

5.1. Principio

TDD “a lo agents” sigue el mismo ciclo Red → Green → Refactor, pero:

- **Red** = AC definidos y fallando (porque la feature aún no está).
- **Green** = implementación mínima para cumplir AC.
- **Refactor** = limpiar el código manteniendo AC en verde, ayudados por `verify`.

5.2. Fases

1. **Red: definir criterio antes de código**
   - Crear / actualizar `.ai/specs/<KEY>-spec.md`.
   - Asegurarse de que los AC:
     - Reflejan lo que el negocio/PO realmente quiere.
     - Son suficientemente concretos para automatizar su verificación.
   - Configurar (si aplica) las pruebas en `.ai/verify/config.yaml` para que al inicio los AC relacionados **fallen**.
2. **Green: implementación mínima**
   - IA + dev trabajan sobre:
     - `AGENTS.md`
     - Spec del ticket
     - Skills relevantes en `.ai/skills/`
   - Desarrollar lo justo para empezar a cumplir AC.
   - Avanzar preferiblemente **AC por AC**.
   - Tras cada ronda significativa de cambios + verify, actualizar `## Execution Result` con el estado de los AC.
3. **Refactor + verify**
   - Refactorizar el código manteniendo los AC.
   - Ejecutar `verify` hasta tener todos los AC en PASS.
   - Dejar constancia en `## Execution Result` de la última verificación en verde.
   - Solo entonces se considera listo para PR.

5.3. TDD clásico (unit tests)

- SDD + TDD “a lo agents” no sustituyen TDD clásico.
- Para lógica crítica o compleja:
  - Tests unitarios (Jest, Vitest, PHPUnit, etc.).
  - Tests de integración / componentes si aplica.

Política recomendada:

- **Obligatorio**: AC claros + verify en verde + `## Execution Result` actualizado.
- **Muy recomendado**: tests unitarios para piezas complejas.

6. Skills de IA del proyecto

> **Objetivo**
>
> Tener un sitio único donde:

- Definir **skills** (tareas repetibles) que queremos que la IA haga siempre igual.
- Reutilizarlos entre devs y proyectos.
- Evitar prompts ad-hoc y contradictorios.

6.1. Dónde viven los skills

Dos niveles:

1. **Skills de proyecto (recomendado)**
   - Carpeta: `.ai/skills/`
   - Un archivo por skill, formato Markdown:

```
.ai/
├── skills/
│   ├── refactor-module.md
│   ├── analyze-bug-report.md
│   └── generate-verify-plan.md
└── ...
```

   - Alternativa mínima: `.ai/skills/README.md` con todos los skills listados.
2. **Skills integrados en la herramienta (Claude Code, etc.)**
   - Carpeta: `.claude/commands/`
   - Son comandos que envuelven uno o varios skills de `.ai/skills/`.
   - Deben:
     - Respetar `AGENTS.md`.
     - Referenciar la spec actual si aplica.
     - Respetar la política de trazabilidad:
       - No marcar tarea como finalizada sin `## Execution Result`.

6.2. Qué es un skill

Un **skill** es una **tarea repetible** que:

- Queremos que la IA ejecute con un protocolo estable.
- Tiene:
  - Objetivo claro.
  - Inputs esperados.
  - Pasos concretos.
  - Output esperado.

Ejemplos de skills típicos:

- Refactorizar un módulo sin romper contratos públicos.
- Analizar un bug report y proponer hipótesis + plan de pruebas.

- Generar un plan de verificación `verify` a partir de una spec.
- Revisar una PR comparando código vs AC de la spec.
- **Actualizar automáticamente la sección** `## Execution Result` tras ejecutar verify.

6.3. Formato recomendado de un skill

Plantilla mínima para `.ai/skills/<nombre-skill>.md`:

```
# Skill: <nombre descriptivo>

## Objetivo
Explicar en 1-3 frases qué hace este skill
y qué problema resuelve.

## Cuándo usarlo
- Lista breve de situaciones en las que tiene sentido usarlo.

## Inputs mínimos
- Qué necesita la IA para poder ejecutar bien el skill:
  - Archivos concretos
  - Enlace o ruta de spec
  - Logs, capturas, etc.

## Pasos que debe seguir la IA
1. Paso 1…
2. Paso 2…
3. ...

## Output esperado
- Qué debe devolver la IA:
  - Tipo de resultado (código, plan, checklist…)
  - Formato (texto estructurado, JSON, markdown, etc.)
```

6.4. Uso de skills por parte de los devs

- Antes de escribir un prompt complejo, comprobar si ya existe un skill:
  - `.ai/skills/`
  - `.claude/commands/`
- Si aparece una tarea repetitiva:
  - Proponer un nuevo skill.
  - Documentarlo siguiendo la plantilla.
  - Enlazarlo desde `AGENTS.md` (sección “Skills disponibles”).
- Para tareas IA-asistidas que ejecutan código y verify:
  - Usar skills que ya integren el patrón:
    - Ejecutar verify.
    - Actualizar `## Execution Result`.
    - Devolver un resumen listo para pegar en la PR si hace falta.

6.5. Relación con SDD y TDD “a lo agents”

- Los skills **no sustituyen** a las specs.
- Los skills **se apoyan** en:
  - `AGENTS.md` como contexto global.
  - Specs en `.ai/specs/` como contexto por ticket.

Ejemplos:

- Skill **“generate-verify-plan”**:
  - Input: `.ai/specs/<KEY>-spec.md`.

- Output: propuesta de escenarios y AC a cubrir en `verify`.
- Skill **“check-spec-compliance”**:
  - Input: diff de la PR + spec.
  - Output: lista de AC cumplidos / no cumplidos.
- Skill **“update-execution-result”**:
  - Inputs:
    - Spec del ticket.
    - Resultado de `verify`.
    - Lista de archivos modificados.
    - Herramientas de IA usadas.
  - Output:
    - Sección `## Execution Result` completa y coherente.

7. Organización del código y carpetas (genérico)

La estructura concreta dependerá del stack, pero los principios son:

- Código organizado por **dominio / módulo**, no solo por tipo de archivo.
- Separar claramente:
  - Lógica de negocio
  - Presentación / UI
  - Infraestructura / acceso a datos

Ejemplo genérico:

```
src/
├── app/                # Entradas de app (Next.js, front, etc.)
├── modules/            # Módulos de dominio
│   ├── users/
│   ├── billing/
│   └── ...
├── lib/                # Utilidades y lógica compartida
├── components/         # Componentes reutilizables de UI
├── api/                # Endpoints API (si aplica)
└── tests/              # Tests (si no van junto al código)
```

> Cada stack (WordPress, Magento, Laravel, Next.js, etc.) tendrá su propia propuesta detallada de estructura, descrita en la plantilla `AGENTS.md` correspondiente.
> Ver plantillas de stack:
> https://iberpixel.atlassian.net/wiki/spaces/HO/pages/1033404418 Request access

8. Buenas prácticas con IA en el día a día

8.1. Antes de empezar a trabajar

1. Asegurarse de que `AGENTS.md` está actualizado.
2. Revisar `CLAUDE.md` y `.claude/settings.json`:
   - Qué carpetas puede leer la IA.
   - Qué rutas están explícitamente excluidas (`.env`, credenciales, etc.).
3. Trabajar siempre en rama de feature, nunca directamente en `main`.

8.2. Durante el desarrollo

- Ser específico: referenciar siempre la spec y los archivos concretos.
- Pedir explicaciones a la IA cuando proponga cambios grandes o no triviales.
- Evitar que la IA:
  - Toque archivos críticos no relacionados con el ticket.
  - Añada dependencias sin revisarlas.
- **Recordar siempre el circuito completo**:
  - Spec → Código → Execution Result → Verify → PR.

8.3. Commits y PR

- Commits atómicos y descriptivos.
- Recomendación para código IA-asistido:
  - Prefijo en el mensaje (`[ai-assisted]`), o
  - Sección explícita en la PR ("Código IA-asistido").
- En toda PR:
  - Enlace a ticket de Jira.
  - Enlace a la spec.
  - Resultado de `verify`.
  - Resumen de cambios relevantes (idealmente relacionado con AC).
  - Confirmación de que `## Execution Result` está al día.

8.4. Convenciones de ramas y commits (Jira + Bitbucket + Canvas)

Ramas Git

- Una rama por **historia de Jira**, no por subtarea.
- Formato recomendado: `feature/<KEY>-descripcion-corta`
  - Ejemplo: `feature/MIM-100-setup-infraestructura`
- Todas las subtareas (`MIM-001` … `MIM-00N`) de esa historia se implementan en la **misma rama**.

Commits

- Formato obligatorio: `<tipo>(KEY): descripción en minúsculas`
  - Ejemplos:
    - `feat(MIM-001): inicializar next.js 14 con typescript y app router`
    - `fix(MIM-003): corregir conexión prisma con variables de entorno`
    - `test(MIM-007): seed script validado con 500 pedidos`
- Tipos permitidos:
  - `feat` , `fix` , `test` , `refactor` , `docs` , `chore`
- `KEY` debe ser el **ID de Jira de la subtarea activa en canvas** (estado **Doing**, color naranja).
- **Nunca** se hace commit si la subtarea no está en estado Doing en el canvas.

Pull Requests

- Siempre desde la rama de la historia:
  - `feature/MIM-100-setup-infraestructura → main`

- Título recomendado:
  - `[MIM-100] Setup & Infraestructura - SalesPulse`
- La descripción de la PR debe incluir:
  - Lista de subtareas cubiertas, con su estado:
    - `- MIM-001 ✅ Next.js 14 + TypeScript`
    - `- MIM-002 ✅ Tailwind + shadcn/ui`
  - Specs relacionadas:
    - `.ai/specs/MIM-100-historia-setup.md`
    - `.ai/specs/MIM-001-spec.md` … `.ai/specs/MIM-005-spec.md`
  - Resultado de `verify`:

    ```
    npm run verify ✅
    ```

  - Sección "Código IA-asistido":
    - Qué subtareas han sido ayudadas por agentes (Claude, etc.).

8.5. Checklist de Cierre de Tarea

Antes de mover cualquier tarea a estado Review, el agente DEBE completar los siguientes tres pasos obligatoriamente:

Paso 1 — Añadir sección `## Execution Result` al spec de la tarea

Fichero: `.ai/specs/<TICKET-ID>-spec.md`

Contenido obligatorio:

Campo	Descripción
Fecha	Fecha de implementación (ISO 8601)
Rama	Rama Git donde se realizó el trabajo
Commit	Hash del commit de implementación
Herramienta IA	Tool y modelo usados (ej: `Claude Code claude-sonnet-4-6`)
Verificación de AC	Tabla: AC-N → PASS/FAIL + breve explicación
Ficheros modificados	Lista de ficheros creados/modificados relevantes
Decisiones técnicas	Cualquier decisión no obvia tomada durante la implementación

---

Paso 2 — Añadir suite de verificación a `.ai/verify/config.yaml`

```
suites:
```

```
  <ticket_id>_<nombre_corto>:
    description: '<Qué verifica esta suite>'
    enabled: true
    related_spec: '.ai/specs/<TICKET-ID>-spec.md'
    verifications:
      - id: AC-N
        name: '<Nombre observable del check>'
        type: filesystem | npm_script | custom | api | playwright
        # filesystem:
        checks:
          - path: <fichero>
            must_exist: true
            must_contain: '<cadena>'
        # npm_script:
        command: npm run <script>
        expected_result: PASS
        exit_code: 0
        timeout: 120
        # custom:
        command: <comando de shell>
        expected_result: PASS | "200" | "true"
        exit_code: 0

```

Tipos soportados: `filesystem`, `npm_script`, `json_check`, `api`, `playwright`, `performance`, `custom`

Paso 3 — Commit de documentación independiente

```
docs(<TICKET-ID>): añadir execution result y suite de verify

Per AGENTS.md § 8.5:
- Sección ## Execution Result añadida al spec de <TICKET-ID>
- Suite <nombre_suite> añadida a .ai/verify/config.yaml

Ref: .ai/specs/<TICKET-ID>-spec.md

```

Justificación

Cada tarea debe tener un registro permanente y verificable de su ejecución para garantizar:

- **Auditoría de cumplimiento** — quién, cuándo y qué se completó
- **Verificación automatizada** — checks de AC reproducibles en CI/CD
- **Traspaso de equipo** — contexto suficiente para que cualquier desarrollador retome el trabajo
- **Análisis de causa raíz** — trazabilidad si surgen problemas tras la implementación

---

> **No negociable:** Ninguna tarea pasa a Review (`finish`) sin estos tres pasos completados.

9. Seguridad y datos

- Nunca exponer a la IA:
  - Credenciales de producción.
  - Datos reales de clientes (PII, financieros, etc.).
- No usar IA para:
  - Generar migraciones directamente sobre BBDD de producción.
  - Manipular backups de datos reales.
- Toda nueva dependencia sugerida por la IA debe evaluarse:
  - Mantenimiento y comunidad.

- Licencia.
- Seguridad (issues conocidos).

10. Checklist rápido para iniciar un proyecto nuevo

1. [ ] Crear repositorio con la estructura base.
2. [ ] Crear / adaptar `AGENTS.md`:
   - Basado en la plantilla global.
   - Añadiendo detalles específicos del stack.
3. [ ] Añadir `CLAUDE.md` y `.claude/settings.json`:
   - Especificar qué debe leer la IA.
   - Excluir `.env`, credenciales y datos sensibles.
4. [ ] Crear estructura `.ai/`:
   - [ ] `specs/`
   - [ ] `verify/config.yaml` (copiando y adaptando ejemplo oficial)
   - [ ] `skills/` (al menos con un par de skills base)
   - [ ] `prompts/` si se necesitan guías de estilo comunes
5. [ ] Configurar `.gitignore`:
   - IA-local, `.env`, ficheros temporales, etc.
6. [ ] Configurar scripts de proyecto (`package.json`, `Makefile`, etc.):
   - `dev`, `test`, `lint`, `format`, `verify`, `build`.
7. [ ] Documentar en `README.md`:
   - Cómo arrancar el proyecto.
   - Cómo usar este framework (enlazar a `AGENTS.md` y esta guía).
8. [ ] **Definir en** `AGENTS.md` la política de trazabilidad IA-asistida:
   - Referencia explícita a esta guía.
   - Reglas de uso de `## Execution Result`.
   - Qué comandos/skills se usan para ejecutar verify y actualizar specs.
9. [ ] **Verificar que al menos un flujo end-to-end Spec → Código → Execution Result → Verify → PR está documentado** en `AGENTS.md`.
10. [ ] **Antes de mover cualquier tarea a Review**:
    - [ ] La spec correspondiente existe en `.ai/specs/<KEY>-spec.md`.
    - [ ] La spec está alineada con el alcance real de la tarea.
    - [ ] La spec incluye sección `## Execution Result` completa y coherente con el código y `verify`.

11. Uso de esta guía y generación de `AGENTS.md`

Este documento en Confluence es la **fuente de verdad corporativa** sobre cómo iniciar y trabajar proyectos con agentes de IA en Iberpixel. Define el estándar común para todos los equipos, pero **no se copia tal cual** en los repositorios.

En cada proyecto deberá existir un archivo `AGENTS.md` en la **raíz del repo**, junto a `README.md`, que:

- Adapta estas directrices al **stack concreto**, contexto y convenciones de ese proyecto.
- Describe los **agentes, skills, límites y flujos reales** que se usan en ese código.

- Sirve como contrato operativo entre el equipo de desarrollo y las herramientas de IA.
- Incluye explícitamente:
  - Que ningún flujo IA-asistido se considera terminado sin actualizar la spec con `## Execution Result`.
  - Cómo deben los agentes usar `.ai/verify/` y documentar los resultados.

Es **válido y recomendable** usar agentes de IA (por ejemplo, Claude) para generar un borrador inicial de `AGENTS.md`, siempre que se cumpla:

- El agente tiene acceso a este documento corporativo.
- El agente puede leer la estructura y el contenido del repositorio del proyecto.
- Un developer revisa, corrige y valida el `AGENTS.md` generado **antes de hacer commit**.

En caso de conflicto entre lo que ponga un `AGENTS.md` de proyecto y esta guía corporativa, **prevalece esta guía**, y el `AGENTS.md` debe actualizarse para alinearse con ella.

12. Dónde ampliar información

- Índice del framework de desarrollo con IA:
  Framework de desarrollo con IA — Índice y punto de entrada
- Documento de kickoff (fundamentos SDD y modelo mental):
  Framework de desarrollo con IA — Documento de kickoff (EOS-244)
- Guía de trabajo con agentes de IA en desarrollo:
  https://iberpixel.atlassian.net/wiki/spaces/HO/pages/1033404418 Request access
- TDD “a lo agents” + verify con Playwright:
  Integración de TDD "a lo agents" en el desarrollo de software con IA
  🧩 Ejemplo listo para desplegar de config.yaml